ADULT INCOME FAIRNESS AUDIT

Machine Learning Project Report

Project Code: EC417
Dataset: UCI Adult Income Dataset
Models: Logistic Regression, Random Forest and CatBoost

Prepared from the supplied project notebook and its saved evaluation outputs.

Contents

1. Abstract

2. Project Objectives

3. Dataset and Features

4. Tools and Technologies

5. Methodology

6. Models

7. Classification Threshold Tuning

8. Test-Set Performance

9. Fairness Audit

10. Error Analysis

11. Bootstrap Confidence Intervals

12. Observations and Discussion

13. Conclusion

14. Reproducibility

1. Abstract

This project develops machine-learning classifiers for the Adult Income dataset and audits whether their predictions differ across demographic groups. The notebook implements Logistic Regression and Random Forest as baseline models, followed by a tuned CatBoost classifier. It also compares predictions made with sex and race available as model inputs against a model trained without those sensitive attributes. Predictive performance is assessed using accuracy, precision, recall, F1 score and ROC-AUC. Fairness is examined using group selection rates and true-positive rates (TPR).

2. Project Objectives

• Load and inspect the Adult Income training and official test datasets.
• Explore income distribution and missing values.
• Preprocess numerical and categorical features.
• Train Logistic Regression, Random Forest and CatBoost models.
• Tune CatBoost hyperparameters and the classification threshold.
• Audit outcomes by sex and race.
• Compare models with and without sensitive attributes.
• Evaluate the final models, analyze errors and estimate confidence intervals.

3. Dataset and Features

The notebook uses the Adult Income dataset with demographic, education, employment and financial attributes. The target is income, mapped to 0 for <=50K and 1 for >50K. The data is split into training and validation subsets using an 80/20 stratified split with random_state=42. The supplied official test dataset is used for final evaluation. The duplicate education-num representation is removed from model inputs in the preprocessing workflow.

4. Tools and Technologies

Python, Jupyter Notebook, NumPy, Pandas, Matplotlib, Seaborn, scikit-learn and CatBoost.

5. Methodology

The workflow consists of dataset loading, exploratory analysis, missing-value handling, feature preparation, stratified train-validation splitting, baseline model training, CatBoost hyperparameter search using RandomizedSearchCV and StratifiedKFold, validation-based threshold selection, fairness calculations, official test evaluation, confusion-matrix and curve analysis, error analysis and bootstrap confidence intervals.

6. Models

Logistic Regression and Random Forest are used as baseline classifiers. CatBoost is then trained with categorical features handled directly after filling missing categorical values. A second CatBoost model excludes sex and race from its input features while retaining those attributes for post-hoc group auditing.

7. Classification Threshold Tuning

The validation precision-recall curve is used to calculate F1 across candidate thresholds. The threshold with the highest validation F1 is selected. The saved validation comparison is:

Metric	Default (0.5)	Tuned threshold
Accuracy	0.8775	0.8736
Precision	0.7943	0.7259
Recall	0.6626	0.7634
F1 Score	0.7225	0.7442

The tuned threshold increases validation recall and F1 while reducing precision and slightly reducing accuracy. Threshold tuning therefore changes the balance between false positives and false negatives; it does not guarantee fairness.

8. Test-Set Performance

The following values are taken from the notebook's saved official test-set evaluation:

Model	Accuracy	Precision	Recall	F1	ROC-AUC
CatBoost – sensitive features, threshold 0.5	0.8748	0.7812	0.6526	0.7111	0.9270
CatBoost – sensitive features, tuned threshold	0.8649	0.7051	0.7361	0.7203	0.9270
CatBoost – without sensitive features, threshold 0.5	0.8748	0.7819	0.6516	0.7108	0.9268

The tuned-threshold CatBoost model has the highest reported test F1 (0.7203) and recall (0.7361), while the default-threshold model has higher accuracy (0.8748) and precision (0.7812). The ROC-AUC values are very similar across the three configurations.

9. Fairness Audit

Selection rate is the proportion of a demographic group predicted as positive. TPR is the proportion of actual positive cases correctly predicted as positive. The gap is calculated as the maximum group value minus the minimum group value. A smaller gap indicates less disparity for that particular metric, but fairness is multidimensional and depends on the application.

Official test-set group metrics – default threshold, by sex:

Group	N	Selection rate	TPR
Female	5,421	0.0806	0.5814
Male	10,860	0.2556	0.6655

Official test-set group metrics – default threshold, by race:

Race group	N	Selection rate	TPR
Amer-Indian-Eskimo	159	0.0755	0.4737
Asian-Pac-Islander	480	0.2417	0.6617
Black	1,561	0.0814	0.5363
Other	135	0.0889	0.4400
White	13,946	0.2112	0.6607

Fairness gaps across model configurations:

Audit	Selection-rate gap	TPR gap
Sex – default threshold	0.1750	0.0842
Race – default threshold	0.1662	0.2217
Sex – tuned threshold	0.2154	0.0747
Race – tuned threshold	0.1977	0.1519
Sex – no sensitive features	0.1687	0.0569
Race – no sensitive features	0.1632	0.1567

Removing sex and race from the model inputs does not eliminate disparities. In this notebook's test results, the selection-rate gaps and TPR gaps change by different amounts across sex and race. This demonstrates that other correlated features can still carry information associated with protected groups. It is not appropriate to conclude that one

model is universally fairer from a single metric.

10. Error Analysis

For the tuned-threshold model, the notebook records 14,082 correct predictions, 1,184 false positives and 1,015 false negatives. The error analysis also groups these outcomes by sex and race. These counts show that both types of classification error occur and that the number of errors should be interpreted alongside each group's total sample size.

11. Bootstrap Confidence Intervals

The notebook uses 1,000 bootstrap iterations with a fixed random generator seed to estimate uncertainty for ROC-AUC and F1. The saved results are:

Model	AUC mean	AUC 95% CI	F1 mean	F1 95% CI
With sensitive features (0.5)	0.9270	0.9227–0.9313	0.7110	0.6987–0.7234
With sensitive features (tuned)	0.9270	0.9227–0.9313	0.7202	0.7090–0.7315
Without sensitive features (0.5)	0.9267	0.9224–0.9310	0.7107	0.6985–0.7225

12. Observations and Discussion

• CatBoost achieves ROC-AUC around 0.927 on the official test set.
• Threshold tuning raises recall and F1 but lowers precision and accuracy in the reported comparison.
• The model without sex and race has performance close to the model using those features, but similar predictive performance does not imply equal group outcomes.
• Selection-rate and TPR gaps differ across sex and race and across thresholds.
• Removing sensitive features alone does not ensure fairness because other variables may act as proxies.
• Bootstrap intervals provide a measure of sampling variability for the reported AUC and F1 estimates.

13. Conclusion

The project demonstrates a complete workflow for income classification and a group-based fairness audit. It compares baseline classifiers and CatBoost, studies the effect of threshold selection, and examines the impact of excluding sensitive attributes. The results show that predictive performance and demographic fairness must be assessed separately. A responsible deployment decision would require defining the relevant fairness criteria, investigating causes of disparities, documenting limitations and considering the social context of the prediction task.

14. Reproducibility

Install the packages listed in requirements.txt, obtain the Adult Income dataset files (adult.data and adult.test) from the dataset source, place them in the paths expected by the notebook, and run the notebook cells in order. The original notebook begins with a Google Colab upload step; for local Jupyter execution, that cell should be replaced with local file-path loading. The exact tuned hyperparameters and threshold are generated during notebook execution and should be preserved in the run outputs.

Note: This report reproduces the metrics stored in the uploaded notebook. It does not invent missing results or claim that the project has been independently rerun.